

AI Pioneers

CCA-F | Module 3

Lab 3.3

From Refinement to Pipeline: **Iterative Workflows & CI/CD Integration**

Scenario: Putting Claude Code on the NorthPeak Refunds Pipeline

Module	M3 — Claude Code Configuration & Workflows
Lab type	Hands-on · Self-paced (Optional) · Run in VS Code, Claude Code & a CI pipeline
Duration	~40 minutes (hands-on)
Environment	Blue Labs VM · VS Code + Claude Code · Python 3.10+ · pytest · GitHub Actions
Sections	S5 (Iterative Refinement Techniques) S6 (CI/CD Integration with Claude Code)

Lab Objectives

By the end of this lab you will be able to:

- **Refine code with test-driven and interview-style loops** — describe new behaviour as a failing test, then iterate write-test → run → implement → run until the whole suite passes.
- **Run Claude Code headless in CI/CD** — use `claude -p` non-interactively in a GitHub Action so Claude reviews every pull request automatically.
- **Emit structured JSON for downstream automation** — have Claude output a `{decision, issues}` object and turn it into a pass/fail gate with `review_gate.py`.

1. Overview

Claude Code is most powerful when it stops being only an interactive helper and becomes part of your pipeline. This lab takes the NorthPeak Outfitters refunds service and moves it along that path. First you refine code with a test-driven loop — the new behaviour is a failing test, and you iterate until it goes green. Then you run Claude headless in CI so a GitHub Action reviews every pull request without anyone watching. Finally you make Claude's output structured JSON and convert it into a pass/fail gate that can block a bad PR. The repo ships green, so everything runs in VS Code right away; the exercises extend it. This lab covers Module 3 sections S5 and S6.

SN	Section	What you will do	Core idea
Ex 1	S5 — Iterative Refinement	Add a restocking-fee rule by writing the failing test first, then making it pass.	The test is the spec; loop red → green.
Ex 2	S6 — CI/CD Integration	Read and run the headless claude -p review the GitHub Action uses on each PR.	Non-interactive Claude as a pipeline step.
Ex 3	S6 — CI/CD Integration	Turn a structured JSON verdict into a pass/fail gate with review_gate.py.	Structured output drives deterministic automation.

2. Scenario

NorthPeak Outfitters’ refunds service decides how much money goes back to a customer — so a wrong number is a real refund error, and every change deserves a test and a review. The team is small and ships often, and they cannot hand-review every pull request. They want two things: a disciplined way to add behaviour (write the test, then the code), and an automated reviewer that runs on every PR and blocks merges that introduce a bug or an untested change.

You will take the refunds service from “works on my machine” to “guarded by a pipeline.” The code already ships green; you extend it with a new rule, then wire Claude into CI as a headless reviewer whose structured verdict becomes a pass/fail gate.

You will:

- Add a restocking-fee rule for opened items using a test-driven loop — failing test first, then the implementation that makes the whole suite pass.
- Read and run the headless claude -p review that the GitHub Action uses on every pull request.
- Convert Claude’s structured JSON verdict into a pass/fail gate that CI uses to block a bad PR.

The pipeline at a glance

```
src/northpeak/refunds.py      → the refund logic you refine (TDD)
.claude/commands/pr-review.md → /pr-review → strict {decision, issues} JSON
.github/workflows/claude-review.yml → headless claude -p on every PR
scripts/review_gate.py        → JSON verdict → exit 0 (pass) / 1 (fail)
```

Why these three together?

They form a chain from change to gate. A test-driven loop produces a change you trust because a test defines “done.” Headless Claude lets that trust scale — a reviewer that runs unattended on every PR. Structured JSON output is the join between them: it turns a review from prose a human reads into data a gate can act on, so “approve” or “request_changes” becomes a deterministic pass/fail. Refine, review automatically, gate on the result.

3. Pre-requisites

3.1 Skilljar Videos — Complete Before the Session

Course	Lessons to watch	Covers
Claude Code in Action	Making changes · Github integration · The Claude Code SDK	S5, S6
Claude Code 101	Code review	S5

Module 3 introduces no new Skilljar courses — these lessons are in courses you completed in Modules 1–2. This lab is optional and assumes you have completed Labs 3.1 and 3.2. The instructor will not re-teach Claude Code basics — bring questions to the Doubt Session instead.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open <https://www.bluelabs.studio/>, locate your VM, click Start, then Connect.
- Get the lab bundle file (lab-3.3-iterative-and-cicd.zip) from your instructor or your Blue Labs VM, then unzip it and open the folder in VS Code; accept the recommended extensions (incl. Claude Code):

```
unzip lab-3.3-iterative-and-cicd.zip
code lab-3.3-iterative-and-cicd
```

- Create a Python environment and install the test dependency:

```
python -m venv .venv && source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt          # pytest>=8.0.0
```

- Confirm the suite is green and the gate works on the samples (no API calls needed):

```
pytest -q                                # expect: 6 passed
python scripts/review_gate.py samples/sample_review.json    # PASS (exit 0)
python scripts/review_gate.py samples/sample_review_fail.json # FAIL (exit 1)
```

Headless mode & your API key

Exercises 1 and 3 need no network — they run `pytest` and the gate locally. Only the live headless review (Exercise 2) calls the API: set `ANTHROPIC_API_KEY` and have Claude Code installed, or push to GitHub and add it as an Actions secret (Settings > Secrets and variables > Actions). CLI flags and JSON shapes can change between versions — `review_gate.py` already handles both a bare review object and the `--output-format json` envelope; check the docs if a flag differs. For the interactive steps (Exercises 1 and 3), run `claude` and sign in when prompted. The bundle is already a git repository with a committed baseline, so `/pr-review`'s git diff sees your changes.

4. Exercises

Do these from the lab folder. The repo starts green; the exercises extend it. Step-by-step tasks are here.

Exercise 1: Add a feature with a test-driven loop (\$5) ~15 min

Background

Test-driven development makes the test the specification. For new behaviour you write a *failing* test first, watch it fail (red), then implement just enough to make it pass (green) — looping write-test → run → implement → run. The project's `CLAUDE.md` sets this as the default working style and adds a hard rule: never weaken or delete a test to go green — fix the code.

New rule to add: if a returned item was *opened*, the refund is reduced by a 15% restocking fee — still only within the 30-day window (outside the window the refund is still 0).

Task

Add the restocking-fee rule by writing the test first, then implementing it without breaking the existing suite.

Step 1 — Write the failing test first

Ask Claude to add the tests and run them, without touching the implementation yet:

```
Write failing tests for a new rule: refund_amount should take an `opened` flag; opened items within the window get 85% of the price (15% restocking fee), and outside the window the refund is still 0. Don't change refunds.py yet — just add the tests and run them so we see them fail.
```

Expected: two new tests — `test_opened_item_restocking_fee` asserting `refund_amount(100.0, 10, opened=True) == 85.0`, plus a paired `test_opened_item_outside_window_still_zero` for the outside-window case (`refund_amount(100.0, 45, opened=True) == 0.0`).

Step 2 — Confirm it fails (red)

Run the suite. The new tests fail because `refund_amount` doesn't accept `opened` yet — which proves the tests actually exercise the new behaviour.

```
pytest -q          # expect: 2 failed, 6 passed (the new tests are RED)
```

Step 3 — Make it pass (green)

Now ask Claude to implement the parameter so the whole suite passes:

```
Now implement the `opened` parameter in refund_amount so all tests pass.
```

The target implementation adds an `opened: bool = False` parameter and a `RESTOCKING_FEE_RATE = 0.15`; defaulting to `False` keeps every existing call and test working:

```
RESTOCKING_FEE_RATE = 0.15

def refund_amount(price, days_since_delivery, opened=False):
    if price < 0:
        raise ValueError("price must not be negative")
    if not within_return_window(days_since_delivery):
```

```
    return 0.0
    if opened:
        return round(price * (1 - RESTOCKING_FEE_RATE), 2)
    return round(price, 2)
```

Step 4 — Confirm green

```
pytest -q          # expect: 8 passed (GREEN)
```

What good looks like

Two new tests were added and failed first (red), then the implementation made the whole suite pass (green) — and the original tests never changed. `opened=False` by default keeps every existing call working, so the new behaviour is additive.

Reflection Questions

- Why write the failing test first instead of implementing and then testing — what does watching it fail (red) actually prove?
- The `opened` parameter defaults to `False`. Why does that matter for the existing tests and callers?
- CLAUDE.md says “never weaken or delete a test to go green — fix the code.” Why is that rule essential to a TDD loop?

Exercise 2: Run Claude headless in CI (S6) ~12 min

Background

Headless (“print”) mode runs Claude once and prints a result, with no interactive session: `claude -p "<prompt>"`. That is exactly what a CI step needs — no REPL, no human in the loop. The workflow in `.github/workflows/claude-review.yml` installs Claude Code, captures the PR diff, runs `claude -p ... --output-format json`, and pipes the result into the gate.

Task

Read the pipeline end to end and run the headless command yourself.

Step 1 — Read the workflow

Open `.github/workflows/claude-review.yml`. Trace the steps: checkout with `fetch-depth: 0` (so the base branch can be diffed), install Claude Code, `git diff origin/<base>...HEAD > pr.diff`, `claude -p ... --output-format json`, then `review_gate.py`.

Step 2 — Run the headless command locally

With `ANTHROPIC_API_KEY` set and Claude Code installed:

```
claude -p "Summarize what src/northpeak/refunds.py does in one sentence." \
--output-format json
```

The result is a JSON *envelope* — the assistant’s text is in the `result` field. That is why the gate unwraps `result`:

```
{
  "type": "result",
  "is_error": false,
  "result": "refund_amount returns the refund for a NorthPeak order ..."
}
```

Step 3 — Use it for real

Push the repo to GitHub and add `ANTHROPIC_API_KEY` as an Actions secret. Open a pull request and the workflow runs the review automatically on the diff.

What good looks like

You can explain each step of the workflow — full-history checkout, install, diff the base branch, headless `claude -p`, gate the JSON — and you understand why the run is non-interactive and why the key is a repo secret, never committed.

Reflection Questions

- What does `claude -p` (headless / print mode) change versus an interactive session, and why is that necessary for CI?
- The workflow diffs `origin/<base>...HEAD` and reviews only the diff, not the whole repo. Why — and why is `fetch-depth: 0` needed?
- The key is supplied as the `ANTHROPIC_API_KEY` Actions secret rather than committed. Why, and what would go wrong otherwise?

Exercise 3: Gate on structured JSON (S6) ~12 min

Background

Machines gate on structure, not prose. The `/pr-review` command (in `.claude/commands/pr-review.md`) tells Claude to output *only* a `{decision, issues}` JSON object; `scripts/review_gate.py` parses it and exits `0` for `approve` or `1` for `request_changes` — the same logic CI uses to block a bad PR.

Task

Run the gate on the samples, then read how it maps a verdict to an exit code.

Step 1 — Run the gate on the provided samples

```
python scripts/review_gate.py samples/sample_review.json          # PASS (exit 0)
python scripts/review_gate.py samples/sample_review_fail.json    # FAIL (exit 1)
echo $?                  # inspect the exit code (PowerShell: $LASTEXITCODE)
```

The fail sample is wrapped in the `--output-format json` envelope (the verdict is a JSON string inside `result`); the gate unwraps it automatically.

Step 2 — Generate a verdict with `/pr-review`

Make a small change, then run `/pr-review` in Claude Code and feed its JSON to the gate:

```
# paste Claude's JSON into review.json, then:
python scripts/review_gate.py review.json
```

Step 3 — Read how the gate works

Open `scripts/review_gate.py`. It strips Markdown fences, unwraps the `result` envelope if present, reads `decision`, and returns an exit code. CI treats a non-zero exit as a failed step — which blocks the merge.

What good looks like

An “approve” verdict passes the gate (exit 0) and a “request_changes” verdict fails it (exit 1) — the exact logic CI uses to block a bad PR. The `/pr-review` command produces the `{decision, issues}` object the gate consumes.

Reflection Questions

- Why have Claude emit a strict `{decision, issues}` JSON object instead of a prose review for the CI gate?
- The gate accepts both a bare review object and the `--output-format json` envelope, and strips code fences. Why build in that tolerance?
- The gate maps `approve` → exit 0 and `request_changes` → exit 1. How does an exit code become a PR gate, and why exit codes specifically?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking back.

#	Question
1	What are the four steps of a test-driven loop, and why must the test fail before you implement?
2	Why does adding a parameter with a default keep an existing suite green, and why does the “never edit a test to go green” rule matter?
3	What does <code>claude -p</code> do, and why is headless mode required for CI?
4	Why review the PR diff (not the whole repo), and why is the API key a secret rather than committed?
5	How does a structured JSON verdict become a pass/fail gate, and why use exit codes?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Implementing first, then writing a test.	A test written after the fact often just confirms what the code does, bugs included. Write it first and watch it fail so you know it tests the new behaviour.

Mistake	Why it matters and what to do instead
Editing a test to make it pass.	That changes the spec to match buggy code. Fix the implementation; never weaken or delete a failing test to go green.
Expecting an interactive session in CI.	CI has no terminal to type into. Use <code>claude -p</code> (headless) so the run is non-interactive and exits with a result.
Committing the API key.	A committed key leaks to anyone with repo access and can't be rotated cleanly. Store it as the <code>ANTHROPIC_API_KEY</code> Actions secret.
Gating on prose instead of JSON.	Free text is ambiguous and fragile to parse. Have Claude emit a strict <code>{decision, issues}</code> object and gate on the decision field.

6. Quick Reference

6.1 The Test-Driven Loop

```
# 1. write the failing test first    (the test is the spec)
# 2. run it -> RED                  pytest -q
# 3. implement just enough          (don't touch other tests)
# 4. run again -> GREEN              pytest -q
# Rule: never weaken/delete a test to go green - fix the code.
```

6.2 Headless Claude (print mode)

```
claude -p "<prompt>" --output-format json      # one-shot, non-interactive
# returns an envelope: { "type":"result", "result":"<assistant text>" }
claude -p "<prompt>" --output-format json --allowedTools "Read"  # CI-safe
```

6.3 The JSON Review Contract

```
# /pr-review tells Claude to output ONLY:
{
  "decision": "approve" | "request_changes",
  "issues": [ { "severity": "blocker"|"warning"|"nit", "message": "..." } ]
}
```

6.4 The Pass/Fail Gate

```
python scripts/review_gate.py samples/sample_review.json      # PASS, exit 0
python scripts/review_gate.py samples/sample_review_fail.json # FAIL, exit 1
claude -p "..." --output-format json | python scripts/review_gate.py
# exit 0 = approve (CI step passes); exit 1 = request_changes (CI blocks merge)
```

6.5 CI/CD Workflow — Shape

Workflow step	What it does
<code>checkout (fetch-depth: 0)</code>	Full history so the base branch can be diffed.
<code>install Claude Code</code>	<code>npm install -g @anthropic-ai/claude-code</code> .
<code>git diff base...HEAD</code>	Capture just the PR's changes for review.
<code>claude -p --output-format json</code>	Headless review; emits the JSON verdict.
<code>review_gate.py</code>	Maps the verdict to exit 0/1 to pass or block the PR.

6.6 File Inventory

File	Section	Purpose
<code>src/northpeak/refunds.py</code>	S5	Refund logic; refined with the TDD loop.
<code>src/tests/test_refunds.py</code>	S5	pytest suite; the failing-test-first target.
<code>.claude/commands/pr-review.md</code>	S6	<code>/pr-review</code> → strict {decision, issues} JSON.
<code>.github/workflows/claude-review.yml</code>	S6	Headless Claude review as a PR gate.
<code>scripts/review_gate.py</code>	S6	JSON verdict → exit 0 (pass) / 1 (fail).
<code>samples/sample_review*.json</code>	S6	Approve / request_changes examples for the gate.
<code>requirements.txt</code>	Setup	Pinned test dependency (pytest>=8.0.0).
<code>CLAUDE.md</code>	S5	TDD working style and the “never weaken a test to go green” rule.

6.7 Further Reading

- Claude Code — headless / print mode & SDK: <https://docs.claude.com/en/docs/claude-code/sdk>
- Claude Code — GitHub Actions / CI: <https://docs.claude.com/en/docs/claude-code/github-actions>
- Claude Code — slash commands: <https://docs.claude.com/en/docs/claude-code/slash-commands>
- Skilljar — Claude Code in Action: Making changes · Github integration · The Claude Code SDK (S5, S6)
- Skilljar — Claude Code 101: Code review (S5)